

 INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA SÃO PAULO	INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO CAMPUS – São Paulo	
Disciplina: SPOWEB1 / AVDI		
Desenv. Web 1 / Artes Visuais	Avaliação Contínua	Data.: 06/05/2026
Nome do Aluno: Davi Medeiros Beserra - Prontuário: SP319325X		
Professora: Claudete de Oliveira		
Github: https://github.com/DaviMedeiros42/AtividadeValidacaoNumeros		

Exercício: Projeto Integrador - Validação de Números com HTML, CSS e JavaScript

Enunciado: Desenvolva uma página web utilizando HTML, CSS e JavaScript capaz de validar números digitados pelo usuário. O sistema deverá analisar o valor informado e exibir mensagens diferentes dependendo da condição atendida.

Raciocínio Passo a Passo:

1. Estrutura da Página (HTML)

Criar a estrutura principal da aplicação utilizando HTML.

A página deverá conter:

- Um título indicando o objetivo do sistema;
- Um campo de entrada numérica;
- Um botão para validar o número;
- Uma área para exibir mensagens ao usuário.

2. Estilização da Interface (CSS)

Aplicar estilos utilizando CSS para melhorar a aparência da página.

A estilização deverá:

- Centralizar os elementos na tela;
- Aplicar cores, bordas e espaçamentos;
- Melhorar a experiência visual do usuário;
- Tornar a interface mais organizada e moderna.

3. Captura do Valor Digitado

Criar uma função JavaScript chamada validarNumero().

Quando o botão for clicado:

- O sistema deverá capturar o valor digitado no campo numérico;
- Também deverá capturar a área onde a mensagem será exibida.

4. Teste de Condição (Campo Vazio)

O sistema deverá verificar inicialmente se o usuário digitou algum valor.

Caminho A:

Se o campo estiver vazio:

- Exibir a mensagem:

Por favor, insira um número.

- A mensagem deverá aparecer na cor vermelha.

5. Conversão do Valor

Caso o campo não esteja vazio:

- O conteúdo digitado deverá ser convertido de texto para número inteiro utilizando parseInt().

6. Teste de Condição (Número Maior que 10)

Caminho A:

Se o número for maior que 10:

O número é maior que 10.

- A mensagem deverá aparecer na cor verde.

7. Teste de Condição (Número Maior que 5)

Caminho B:

Caso o número não seja maior que 10, o sistema deverá verificar se ele é maior que 5.

Se for verdadeiro:

O número é maior que 5, mas menor ou igual a 10.

- A mensagem deverá aparecer na cor laranja.

8. Teste de Condição Final

Caminho C:

Caso nenhuma condição anterior seja atendida:

O número é 5 ou menor.

- A mensagem deverá aparecer na cor azul.

9. Finalização

Após a verificação:

- O sistema exibirá o resultado correspondente;
- O usuário poderá testar novos valores livremente;
- O processo poderá ser repetido quantas vezes desejar.

Solução

Código HTML

```
<!DOCTYPE html>
<html lang="pt-br">

<head>

  <!--
    Define o conjunto de caracteres utilizado no documento.
    UTF-8 permite exibir corretamente acentos e caracteres especiais.
  -->
  <meta charset="UTF-8">

  <!--
    Configuração de responsividade da página.
    Faz com que o layout se adapte em celulares e tablets.
  -->
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!--
  Título exibido na aba do navegador.
-->
<title>Exemplo Exercício Desvio Condicional – Validação Número</title>

<!--
  Importação do arquivo CSS responsável
  pela estilização visual da página.
-->
<link rel="stylesheet" href="styles.css">

</head>

<body>

  <!--
    Container principal da aplicação.
    Utilizado para agrupar todos os elementos visuais.
  -->
  <div class="container">

    <!--
      Título principal da página.
      Destaca o objetivo do sistema.
    -->
    <h1>Validação de Números</h1>

    <!--
      Campo de entrada numérica.
      O usuário deverá informar um número para validação.
    -->
    <input
      type="number"
      id="numero"
      placeholder="Digite um número"
    >

    <!--
      Botão responsável por executar a função validarNumero().
      O evento onclick chama a função JavaScript ao clicar.
    -->
    <button onclick="validarNumero()">
      Validar
    </button>

    <!--
      Área destinada à exibição das mensagens

```

```
        de validação para o usuário.
    -->
    <p id="mensagem"></p>

</div>

<!--
    Importação do arquivo JavaScript.
    O script é carregado ao final do body
    para melhorar o carregamento da página.
-->
<script src="script.js"></script>

</body>
</html>
```

Código CSS

```
/*
    Configuração geral da página.
    Remove margens padrão do navegador e
    centraliza o conteúdo horizontal e verticalmente.
*/
body {

    margin: 0;
    padding: 0;
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;

    display: flex;
    justify-content: center;
    align-items: center;

    height: 100vh;

}

/*
    Estilização do container principal.
    Utiliza o Box Model para criar espaçamento,
    bordas arredondadas e sombra visual.
*/
.container {

    background-color: white;

    padding: 30px;
```

```
border-radius: 10px;

box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.2);

text-align: center;

width: 300px;

}

/*
  Estilo aplicado ao título principal.
*/
h1 {

  color: #333;

  margin-bottom: 20px;

}

/*
  Estilização do campo de entrada numérica.
*/
input {

  width: 100%;

  padding: 10px;

  margin-bottom: 15px;

  border: 1px solid #ccc;

  border-radius: 5px;

  box-sizing: border-box;

}

/*
  Estilização do botão de validação.
*/
button {

  width: 100%;

  padding: 10px;
```

```

background-color: #333;

color: white;

border: none;

border-radius: 5px;

cursor: pointer;

font-size: 16px;

}

/*
  Efeito visual aplicado quando o usuário
  passa o mouse sobre o botão.
*/
button:hover {

  background-color: #555;

}

/*
  Estilização da área de mensagens.
*/
#mensagem {

  margin-top: 20px;

  font-weight: bold;

}

/*
  Responsividade básica da interface.
  Ajusta o tamanho do container em telas menores.
*/
@media (max-width: 768px) {

  .container {

    width: 90%;

  }

}

```

Código Javascript

```
/*
  Função responsável por validar o número
  informado pelo usuário no campo de entrada.
*/
function validarNumero() {

  /*
    Captura o valor digitado no campo
    identificado pelo id "numero".
  */
  let numero = document.getElementById("numero").value;

  /*
    Captura o elemento responsável
    pela exibição das mensagens.
  */
  let mensagem = document.getElementById("mensagem");

  /*
    Verifica se o campo está vazio.
  */
  if (numero === "") {

    /*
      Exibe mensagem de alerta ao usuário
      solicitando o preenchimento do campo.
    */
    mensagem.textContent = "Por favor, insira um número.";

    /*
      Define a cor vermelha para indicar erro.
    */
    mensagem.style.color = "red";

  } else {

    /*
      Converte o conteúdo digitado
      de texto para número inteiro.
    */
    numero = parseInt(numero);

    /*
      Verifica se o número é maior que 10.
    */
    if (numero > 10) {
```

```

    mensagem.textContent = "O número é maior que 10.";

    mensagem.style.color = "green";

} else {

    /*
     Verifica se o número é maior que 5
     e menor ou igual a 10.
    */
    if (numero > 5) {

        mensagem.textContent =
            "O número é maior que 5, mas menor ou igual a 10.";

        mensagem.style.color = "orange";

    } else {

        /*
         Caso nenhuma condição anterior seja verdadeira,
         o número será considerado menor ou igual a 5.
        */
        mensagem.textContent = "O número é 5 ou menor.";

        mensagem.style.color = "blue";

    }

}

}

}

```

Explicação Técnica

Neste projeto foram utilizados conceitos fundamentais do desenvolvimento web:

- HTML para estruturar os elementos da página;
- CSS para estilizar a interface;
- JavaScript para implementar a lógica de validação.

A estrutura condicional if / else foi utilizada para verificar diferentes situações envolvendo o número digitado pelo usuário.

O método `getElementById()` permitiu capturar elementos da página HTML, enquanto `textContent` e `style.color` foram usados para alterar dinamicamente o conteúdo e a aparência das mensagens exibidas.

Além disso, a função `parseInt()` foi utilizada para converter o valor digitado de texto para número inteiro antes das comparações lógicas.

Conclusão

A atividade permitiu reforçar conceitos importantes de programação e desenvolvimento web, como:

- Manipulação de elementos HTML com JavaScript;
- Estruturas condicionais;
- Captura de dados digitados pelo usuário;
- Estilização com CSS;
- Organização da lógica de programação.

O exercício também contribuiu para o entendimento da integração entre HTML, CSS e JavaScript na construção de páginas interativas e dinâmicas.

Referências Bibliográficas

1. DEITEL, Paul; DEITEL, Harvey. JavaScript: guia do programador. 7. ed. Porto Alegre: Bookman, 2013.
2. SILVA, Maurício Samy. HTML5: a linguagem de marcação que revolucionou a web. São Paulo: Novatec Editora, 2011.